```
*/
snprintf(data_line,
sizeof(data_line),
"%02u:%02u:%02u %02u/%02u/%04u ; %u ; %s",
date_time.hour,
date_time.minute,
date_time.second,
date_time.day,
date_time.month,
date_time.year,
light_value,
gps_data);

/*
* Enregistrement de la ligne dans le fichier de mesures.
*/
if (SDLogger_WriteLine("datalog.txt", data_line) != HAL_OK)
{
printf("Erreur lors de l'écriture dans datalog.txt.\r\n");
}
else
{
/*
* Affichage de la même ligne dans le terminal série.
*/
printf("%s\r\n", data_line);
}

HAL_Delay(1000);
}
}
```

Le reste de l'équipe regarde le code, incrédule...

Ça y est, j'ai un super programme ! J'ai réussi à récupérer les informations provenant des principaux composants de la station :

Le GPS m'a tout de même posé quelques difficultés. Il communique à l'aide d'une liaison série UART, mais nous utilisons également une liaison série pour configurer le système et afficher des informations dans le terminal.

Lors de mes premiers essais, les données envoyées par le GPS se mélangeaient avec les messages destinés à l'utilisateur. L'interface devenait difficile à comprendre : les trames de géolocalisation apparaissaient au milieu des paramètres et des messages de configuration.

Il n'est donc pas envisageable d'utiliser la même interface série pour le GPS et pour la communication avec l'utilisateur.

Heureusement, le microcontrôleur STM32L476RG possède plusieurs périphériques UART. J'ai donc utilisé :

- USART1 pour communiquer avec le GPS ;
- USART2 pour communiquer avec le terminal par l'intermédiaire du port série virtuel du ST-LINK.

Pour le capteur de luminosité, connecté à une entrée analogique, je me contente pour le moment d'enregistrer la valeur mesurée, sans effectuer de traitement particulier.

La carte SD communique avec le microcontrôleur par l'intermédiaire du bus SPI. Je parviens désormais à créer un fichier et à y enregistrer les différentes données.

J'ai ainsi pu identifier les périphériques et les bibliothèques nécessaires. Voilà à quoi ressemble le programme :

Code source de main.c :

```
/*
* LED utilisateur LD2 de la carte NUCLEO-L476RG :
* PA5, active à l'état haut.
*/
#define LED_GPIO_PORT GPIOA
#define LED_PIN GPIO_PIN_5

/*
* Bouton utilisateur B1 de la carte NUCLEO-L476RG :
* PC13, actif à l'état bas.
*/
#define BUTTON_GPIO_PORT GPIOC
```